

Laporan Praktikum DevOps

Linux Foundation for DevOps:
Shell Scripting, CLI, & Manajemen Sistem

Nama Mahasiswa	Cikal Pratama
NIM	4332511053
Mata Kuliah	DevOps (RKS305) — 3 SKS
Program Studi / Semester	Rekayasa Keamanan Siber / 3
Dosen Pengampu	Antoni Haikal
Prasyarat	JS-DVO-01 (Minggu 1) — selesai
Berkas	M02_4332511053_Cikal Pratama.pdf

Daftar Isi

1. Tujuan & Ringkasan Capaian
2. Dasar Teori Ringkas (FHS, Izin, Pipe, Exit Code)
3. Hasil Pelaksanaan
 1. JOB 1 — Struktur Berkas, Kepemilikan & Izin Akses
 2. JOB 2 — Analisis Log dengan Pipeline Teks
 3. JOB 3 — Proses, Layanan & Sumber Daya Sistem
 4. JOB 4 — Skrip Bash Modular `sysreport.sh`
 5. JOB 5 — Refaktor `setup.sh` Menjadi Modular
4. Bonus Challenge — "Health Watcher"
5. Jawaban Pertanyaan Analisis (4.1)
6. Kesimpulan
7. Lampiran (kode skrip, bukti output, repo & screenshot)

Catatan keterbukaan. Seluruh keluaran terminal pada laporan ini diambil dari eksekusi nyata skrip yang disertakan. Angka spesifik (sebaran status, CFR, pemakaian disk/memori) bergantung pada data sintetis acak dan mesin eksekusi, sehingga dapat berbeda saat Anda menjalankannya. Lingkungan eksekusi berjalan sebagai `root` ; pada workstation mahasiswa nilai `id` /pemilik berkas menyesuaikan akun Anda. Tautan repository & tangkapan layar pada Lampiran diisi mahasiswa.

1. Tujuan & Ringkasan Capaian

Melanjutkan layanan PT Sentra Digital Batam dari Minggu 1: layanan kini berjalan di server dan sebagian pengguna menerima galat, namun belum ada perkakas pemantauan. Berbekal akses CLI dan sebuah berkas log, praktikum ini menelusuri penyebabnya, memeriksa kesehatan sistem, lalu mengubah prosedur manual menjadi skrip Bash modular yang dapat dijalankan siapa pun dengan hasil sama.

Kode	Rumusan	Status
Sub-CPMK 2.1	FHS, model kepemilikan/izin, prinsip least privilege.	Tercapai (JOB 1)
Sub-CPMK 2.2	grep/awk/sed/sort/uniq/find via pipe & redirection untuk analisis log.	Tercapai (JOB 2)
Sub-CPMK 2.3	Diagnosis sistem (ps/top/df/free/ss/systemctl) & kendali proses aman.	Tercapai (JOB 3)
Sub-CPMK 2.4	Skrip Bash modular: fungsi, getopts, validasi, exit code bermakna.	Tercapai (JOB 4)
Sub-CPMK 2.5	Refaktor <code>setup.sh</code> berpustaka bersama + logging + shellcheck.	Tercapai (JOB 5)

Tiga artefak wajib berjalan dan lolos `shellcheck` tanpa peringatan: `buat-log.sh`, `sysreport.sh`, dan `setup.sh` versi modular (ditambah bonus `healthwatch.sh`).

2. Dasar Teori Ringkas

Linux adalah substrat kerja DevOps: Docker, Kubernetes, Terraform, Ansible, hingga runner CI/CD berjalan native di atas kernel Linux dan dikelola lewat CLI, bukan GUI. Dua pilar interaksinya:

- Filesystem Hierarchy Standard (FHS) — satu pohon tunggal berakar `/`. `/etc` (konfigurasi), `/var/log` (log — sumber observability), `/usr/bin` (biner CLI), `/tmp` (artefak sementara CI/CD). Standarisasi inilah yang membuat skrip berpindah dari laptop ke server/kontainer tanpa mengubah path.
- Ownership & Permissions — entitas User/Group/Other dengan izin `r(4)/w(2)/x(1)`. Lahir prinsip Least Privilege: beri izin seminimal mungkin. Anti-pola berbahaya: `chmod 777` dan menjalankan aplikasi/kontainer sebagai `root`.

Filosofi Unix "do one thing and do it well" + operator aliran data: pipe (`|`), redirection (`>` timpa, `>>` tambah), dan stream merging (`>&1`) menyambung program kecil menjadi "mesin analitik" log.

Kontrak Exit Code: `0` = sukses; non-nol = gagal. Runner CI/CD tidak melihat tampilan — mereka membaca exit code. Anti-pola "membutakan pipeline" (`|| true` atau memaksa `exit 0`) membuat runner menganggap semua lulus meski tes gagal. Karena itu `set -euo pipefail` + `trap`, pustaka fungsi bersama,

logging terstruktur, idempotensi, dan analisis statis menjadikan shell script bentuk paling awal Infrastructure as Code.

Tabel 2.1 — Peta Perkakas CLI dalam Alur Kerja DevOps

Kelompok	Perkakas Inti	Peran
Navigasi & berkas	ls, cd, stat, find, du	Menelusuri artefak build, menemukan konfigurasi & log
Izin & kepemilikan	chmod, chown, umask, id	Menerapkan least privilege pada skrip, kunci, direktori
Pengolah teks	grep, awk, sed, cut, sort, uniq	Analisis log, ekstraksi metrik, penyaringan keluaran
Proses	ps, top, pgrep, kill, jobs	Mengendalikan aplikasi; penghentian anggun
Sumber daya	df, free, uptime, ss	Diagnosis kapasitas & konflik port
Layanan & log	systemctl, journalctl	Mengelola daemon & menelusuri kegagalan
Skrip & kualitas	bash, getopts, trap, shellcheck	Prosedur sebagai kode yang dapat diuji & ditinjau

3. Hasil Pelaksanaan

3.1 JOB 1 — Struktur Berkas, Kepemilikan & Izin Akses

Identitas, `umask`, dan percobaan izin pada berkas yang mensimulasikan kredensial (keluaran nyata):

```
$ id
uid=0(root) gid=0(root) groups=0(root)
$ umask
0022
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
overlay          95G   19G   77G   20% /

--- percobaan izin akses ---
$ stat -c '%a %U:%G %n' kredensial-latihan.txt (default, dipengaruhi umask 0022)
644 root:root kredensial-latihan.txt
$ chmod 600 ; stat ... -> 600 root:root (least privilege)
$ chmod u+x,go-rwx ; stat ... -> 700 root:root (notasi simbolik)
$ chmod 600 (kembalikan aman) -> 600 root:root
```

Interpretasi digit izin. 644 = rw-r--r-- (pemilik baca+tulis; grup & other hanya baca). 600 = rw----- (hanya pemilik; grup & other kehilangan akses baca — tepat untuk berkas kredensial). umask 0022 menjelaskan mengapa berkas baru tidak pernah langsung punya izin eksekusi: bit x di-mask, selaras least privilege.

Catatan: lingkungan eksekusi berjalan sebagai `root`; pada mesin mahasiswa `id` & pemilik berkas akan menampilkan akun Anda (mis. `cikal`).

3.2 JOB 2 — Analisis Log dengan Pipeline Teks

Generator `buat-log.sh` (Lampiran B.1) menghasilkan `app.log` 500 baris berformat Common Log sehingga kolom konsisten: `$7` =endpoint, `$9` =kode status, `$10` =waktu respons (ms). Verifikasi bentuk data: `awk '{print NF}' | sort -u` → 10 kolom per baris. Contoh baris:

```
192.168.5.20 - - [03/Oct/2026:15:42:09 +0000] "GET /api/users HTTP/1.1" 500 381
```

Sebaran kode status — `awk '{print $9}' | sort | uniq -c | sort -rn`

```
205 200
97 404
59 301
56 502
47 201
36 500
```

Endpoint paling sering gagal (≥500) — `awk '$9>=500{print $7}' | sort | uniq -c | sort -rn`

```
30 /health
23 /api/users
21 /static/app.js
18 /api/orders
```

Metrik performa & proksi Change Failure Rate

Rata-rata respons: 613.7 ms dari 500 permintaan
Proksi Change Failure Rate: 18.40% (92 dari 500) # 92 = jumlah 500 (36) + 502 (56)

Kegagalan (≥500) tersebar di semua endpoint dengan konsentrasi tertinggi di `/health` (30) dan `/api/users` (23). Hasil ringkas disimpan ke `ringkasan-log.txt` (Lampiran E) via `{ ... } | tee`. Kaitan DORA: angka 18,40% adalah proksi Change Failure Rate dari Minggu 1.

3.3 JOB 3 — Proses, Layanan & Sumber Daya Sistem

Aplikasi Minggu 1 dijalankan sebagai proses latar, ditemukan kembali, diverifikasi port-nya, lalu dihentikan secara anggun (SIGTERM setelah verifikasi PID). Keluaran nyata:

```
$ ps -eo pid,ppid,user,%cpu,%mem,etime,comm --sort=-%cpu | head -n 8
  PID   PPID USER   %CPU %MEM   ELAPSED COMMAND
  2373   2371 root     5.0  0.1    00:02 python3      <- aplikasi kita
    219    206 root     1.4  1.5    15:15 node
     10      7 root     0.8  0.6    15:19 uvicorn

$ pgrep -af 'src/app.py'
2373 python3 src/app.py
$ ss -ltnp | grep 5000
LISTEN 0 128 127.0.0.1:5000 0.0.0.0:* users:(("python3",pid=2373,fd=3))
$ curl -s http://127.0.0.1:5000/health
{"status":"ok"}

--- penghentian anggun ---
$ ps -p 2373 -o pid,comm,etime      # WAJIB verifikasi sebelum sinyal
  2373 python3          00:02
$ kill -TERM 2373
Aplikasi berhasil dihentikan secara anggun.
$ ss -ltn | grep 5000              # setelah mati
(port 5000 sudah tidak LISTEN)
```

Kondisi sumber daya

```
$ df -h /
overlay 95G 19G used 77G avail 20%
$ free -h
Mem: 15Gi total, 10Gi used, 4.9Gi available (≈67% terpakai)
$ uptime
up 1 day, 18:02, load average: 0.60, 0.88, 0.97
```

SIGTERM vs SIGKILL. Aplikasi berhenti ≤ 2 detik setelah SIGTERM — sempat menutup koneksi & menyimpan data. SIGKILL (-9) tidak memberi kesempatan itu sehingga berisiko merusak berkas; dipakai hanya bila SIGTERM tak direspons. Baris LISTEN :5000 yang hilang dari ss membuktikan proses benar-benar berhenti — cara cepat mendiagnosis konflik port. (Pada lingkungan container ini systemd tidak aktif, sesuai catatan job sheet untuk WSL2.)

3.4 JOB 4 — Skrip Bash Modular `sysreport.sh`

Prosedur pemeriksaan JOB 3 diubah menjadi skrip (Lampiran B.2) yang memuat fungsi (`disk_usage_pct` , `mem_used_pct` , `proc_count`), penguraian argumen `getopts` , validasi masukan (ambang harus angka), serta exit code bermakna. Contoh keluaran JSON (`-j`):

```
{"host":"agent-env-...", "disk_pct":20, "mem_pct":67, "proc":23, "status":"OK"}
```

Tabel 3.1 — Pengujian seluruh jalur & exit code (hasil nyata)

Skenario	Perintah	Exit code	Makna
Normal	<code>./sysreport.sh</code>	0	Sistem sehat
JSON	<code>./sysreport.sh -j</code>	0	Sehat, format JSON
Lewat ambang	<code>./sysreport.sh -d 1</code>	2	Disk $\geq$ ambang → PERINGATAN
Validasi masukan	<code>./sysreport.sh -d abc</code>	1	Ambang bukan angka → galat
Opsi tak dikenal	<code>./sysreport.sh -z</code>	1	Galat penggunaan
Bantuan	<code>./sysreport.sh -h</code>	0	Cetak usage lalu keluar

Mengapa perbedaan ini mutlak bagi CI/CD? Runner hanya membaca exit code. Perbedaan 0 (lanjut), 2 (kondisi peringatan yang dapat ditangani berbeda), dan 1 (galat penggunaan) memungkinkan pipeline mengambil keputusan tepat alih-alih "buta". Analisis statis: `shellcheck sysreport.sh` → bersih, tanpa peringatan.

3.5 JOB 5 — Refaktor `setup.sh` Menjadi Modular

Fungsi bersama dipisah ke `lib/common.sh` (Lampiran B.4): logging bertimestamp (`log_info/log_warn/log_error`), `die`, `require_cmd`, `port_is_free`. `setup.sh` (Lampiran B.3) kini `source` pustaka itu dan mengganti seluruh `echo` dengan fungsi logging. Keluaran nyata + isi `/tmp/sentra-deploy.log` :

```
2026-10-04T07:22:53+0000 [INFO ] [1/5] Memeriksa prasyarat...
2026-10-04T07:22:55+0000 [INFO ] [2/5] Menyiapkan virtual environment...
2026-10-04T07:22:59+0000 [INFO ] [3/5] Memasang dependensi terkunci...
2026-10-04T07:22:59+0000 [INFO ] [4/5] Menjalankan aplikasi pada port 5000...
2026-10-04T07:22:59+0000 [INFO ] [5/5] Melakukan smoke test...
2026-10-04T07:23:01+0000 [INFO ] SUKSES: aplikasi berjalan dan lulus health check (PID 1486, port 5000).
```

Dibanding Minggu 1, log kini memuat stempel waktu ISO-8601 & level ([INFO]/[WARN]/[ERROR]) yang tersimpan permanen di berkas — mempercepat penelusuran insiden (kapan tiap tahap terjadi, di mana melambat).
`shellcheck setup.sh lib/common.sh` → bersih.

4. Bonus Challenge — "Health Watcher"

Skrip `healthwatch.sh` (Lampiran B.5) memantau endpoint `/health`. Pengujian wajib dijalankan: pantau → hentikan aplikasi (`kill -TERM`) → tunggu → hidupkan lagi → tekan Ctrl+C. Keluaran nyata:

```
Memantau http://127.0.0.1:5000/health setiap 2s - tekan Ctrl+C untuk berhenti.
2026-10-04T07:26:09+0000 status=UP    rt_ms=13
2026-10-04T07:26:13+0000 status=UP    rt_ms=14
2026-10-04T07:26:15+0000 status=DOWN  rt_ms=12
!! INSIDEN #1 dimulai pada 2026-10-04T07:26:15+0000
2026-10-04T07:26:19+0000 status=DOWN  rt_ms=12
2026-10-04T07:26:21+0000 status=UP    rt_ms=13
>> PULIH pada 2026-10-04T07:26:21+0000 - MTTR insiden ini: 6 detik
2026-10-04T07:26:25+0000 status=UP    rt_ms=14

===== RINGKASAN SESI HEALTH WATCHER =====
Jumlah pemeriksaan      : 9
Jumlah insiden          : 1
Total waktu padam       : 6 detik
Ketersediaan           : 66.67%
Berkas log              : /tmp/healthwatch.log
=====
exit code = 2 (minimal satu insiden terjadi)
```

#	Ketentuan	Status
1	--interval N (def 5) & --url (def /health) + validasi argumen	✓
2	Log terstruktur 1 baris/cek: timestamp ISO-8601, UP/DOWN, rt_ms	✓
3	Deteksi transisi + hitung MTTR saat kembali UP	✓ (MTTR 6 dtk)
4	<code>trap SIGINT</code> → ringkasan sesi (cek, insiden, padam, %availability)	✓
5	Exit code bermakna: 0 tanpa insiden, 2 bila ada insiden	✓ (exit 2)
6	Lolos <code>shellcheck</code> tanpa peringatan	✓

[Tempel tangkapan layar ringkasan sesi Health Watcher di sini]

Kumpulkan sebagai commit terpisah: `feat(challenge): health watcher dengan MTTR dan ringkasan sesi` (nilai tambahan maksimal +10).

5. Jawaban Pertanyaan Analisis (4.1)

5.1 Batas kemampuan `set -e`

Meski skrip memuat `set -euo pipefail`, perintah gagal tidak selalu menghentikan eksekusi. Minimal dua situasi:

1. Di dalam kondisi `if / while` atau sisi kiri `&& / ||`. Status gagal di konteks ini "dikonsumsi" sebagai nilai boolean, jadi `set -e` diabaikan. Contoh: `if grep xyz f; then ...` tidak berhenti walau `grep` gagal. Penanganan: tangani eksplisit — `if ! cmd; then die "..."; fi`, sehingga kegagalan jadi keputusan sadar, bukan lolos diam-diam.
2. Pada command substitution di penugasan variabel, mis. `out="$(cmd_gagal)"`. Pada banyak kondisi, status keluar penugasan = status `local /assignment`, bukan `cmd`, sehingga skrip jalan terus dengan `out` kosong. Penanganan: pisahkan deklarasi & penugasan lalu periksa — `local out; out="$(cmd)" || die "gagal"` (pola ini juga alasan `shellcheck` SC2155 menyarankan memisah `declare` dari assignment, seperti yang saya terapkan pada `sysreport.sh`).

Situasi lain: perintah di dalam pipe selain yang terakhir (ditangani `pipefail`), dan fungsi yang dipanggil di konteks kondisi.

5.2 Agregat yang menyesatkan

Proksi Change Failure Rate = 18,40% (92 dari 500). Angka agregat tunggal ini menyesatkan karena menyamarkan sebaran per endpoint: dari JOB 2, kegagalan ≥ 500 terkonsentrasi di `/health` (30) dan `/api/users` (23), sementara `/api/orders` (18) relatif lebih rendah. Jika tim hanya melihat "18%", mereka tidak tahu harus memperbaiki yang mana lebih dulu. Metrik tambahan yang lebih berguna: failure rate per endpoint = $(\text{jumlah } \geq 500 \text{ pada endpoint} \div \text{total permintaan endpoint}) \times 100\%$ — mengurutkan prioritas perbaikan ke endpoint dengan rasio gagal tertinggi relatif terhadap trafiknya. (Alternatif lain: error-count berbobot latensi, agar endpoint lambat dan gagal naik prioritas.)

5.3 CLI versus GUI

CLI menjadi prasyarat otomasi karena tiga sifat:

- Reproducibility — perintah/skrip memberi hasil sama di mesin mana pun (berkat FHS & exit code), berbeda dengan klik GUI yang sulit diulang persis.
- Auditability — setiap perintah terekam (history, log terstruktur, diff commit); siapa mengubah apa dapat ditelusuri.
- Version control — skrip adalah teks yang bisa di-diff, ditinjau via pull request, dan di-rollback — fondasi Infrastructure as Code.

Situasi GUI tetap lebih tepat: eksplorasi/analisis visual data yang belum terstruktur — mis. membaca dashboard observability (Grafana) untuk mengenali pola anomali lintas waktu, atau desain antarmuka. Di sini persepsi visual manusia lebih cepat daripada membaca angka mentah di terminal.

5.4 Least privilege pada otomasi

Saran menjalankan `setup.sh` dengan `sudo` + memberi `chmod 777` pada seluruh direktori aplikasi berbahaya:

- Risiko `777` : setiap pengguna/proses (termasuk malware yang berhasil masuk) dapat menulis ulang kode sumber, membaca berkas kredensial, atau menyisipkan backdoor — pelanggaran total least privilege.
- Risiko `sudo /root`: bila aplikasi punya celah (mis. RCE), penyerang otomatis memperoleh hak setingkat administrator atas host.

Dua mekanisme pengganti yang menyelesaikan "Permission denied" tanpa menaikkan hak:

1. Kepemilikan yang benar + izin minimal — jadikan user aplikasi sebagai pemilik direktori lalu `chmod 750` (atau `700` untuk berkas rahasia `600`). Masalah akses selesai tanpa membuka ke "Other".
2. Isolasi lingkungan per-pengguna — gunakan virtual environment di direktori milik user (seperti `.venv` pada skrip ini) sehingga instalasi dependensi tidak butuh akses global/ `sudo` sama sekali (menghindari galat `externally-managed-environment`).

5.5 Idempotensi

Masing-masing skrip dijalankan tiga kali berturut-turut:

Skrip	Idempotent?	Bukti/alasan
<code>sysreport.sh</code>	YA	Bersifat read-only (hanya membaca <code>df/free/ps</code>). Tiga kali eksekusi memberi struktur keluaran & exit code sama (0); tidak mengubah state sistem.
<code>setup.sh</code>	Sebagian / terjaga	Pembuatan <code>venv</code> & instalasi dipandu guard (<code>[-d .venv] </code> , pin versi) sehingga aman diulang. Namun langkah "jalankan app" mengikat port; eksekusi kedua saat app masih hidup gagal cepat lewat <code>port_is_free ... die</code> — gagal terkendali, bukan merusak state.

Idempotensi menjadi syarat mutlak pada Infrastructure as Code (minggu ke-12): tool seperti Ansible/ Terraform menjalankan deklarasi berulang kali dan harus konvergen ke state yang sama tanpa efek samping ganda. Skrip yang tidak idempotent akan menggandakan sumber daya atau merusak konfigurasi saat dijalankan ulang oleh pipeline.

6. Kesimpulan

- Penyebab galat ditemukan murni dari CLI: kegagalan ≥ 500 sebesar 18,40% terkonsentrasi di `/health` & `/api/users` (JOB 2) — tanpa dashboard apa pun.
- Proses didiagnosis & dihentikan anggun (SIGTERM setelah verifikasi PID); hilangnya `LISTEN :5000` membuktikan penghentian (JOB 3).
- Prosedur manual diubah menjadi skrip modular dengan fungsi, `getopts`, validasi, & exit code bermakna (0/1/2), lolos `shellcheck` (JOB 4).
- `setup.sh` direfaktor berpustaka `lib/common.sh` + logging bertimestamp — bentuk awal Infrastructure as Code (JOB 5).
- Bonus Health Watcher memantau ketersediaan & menghitung MTTR (6 detik, availability 66,67%, exit 2) — menyambung metrik DORA Minggu 1.

7. Lampiran

Lampiran B.1 — `buat-log.sh`

```
#!/usr/bin/env bash
# buat-log.sh - Generator log sintetis (reproducible) untuk latihan analisis.
set -euo pipefail
OUT="{1:-app.log}"
IPS=(10.10.0.11 10.10.0.12 10.10.0.13 172.16.4.7 192.168.5.20)
PATHS=(/health /api/orders /api/users /static/app.js)
CODES=(200 200 200 200 201 301 404 404 500 502)
: > "$OUT"
for _ in $(seq 1 500); do
    ts=$(date -d "$((RANDOM % 1440)) minutes" '+%d/%b/%Y:%H:%M:%S %z')
    ip=${IPS[$((RANDOM % ${#IPS[@]}))]}
    p=${PATHS[$((RANDOM % ${#PATHS[@]}))]}
    c=${CODES[$((RANDOM % ${#CODES[@]}))]}
    ms=$((RANDOM % 1200 + 5))
    echo "$ip - - [$ts] \"GET $p HTTP/1.1\" $c $ms" >> "$OUT"
done
echo "Dibuat: $OUT ($(wc -l < "$OUT") baris)"
```

Lampiran B.2 — sysreport.sh

```
#!/usr/bin/env bash
# sysreport.sh - Laporan kesehatan sistem ringkas untuk kebutuhan operasional.
set -euo pipefail
SCRIPT_NAME="$(basename "$0")"; readonly SCRIPT_NAME
readonly VERSION="1.0.0"
THRESHOLD_DISK=80
FORMAT="text"
log() { printf "[%s] %s\n" "$(date '+%H:%M:%S')" "$*" >&2; }
die() { log "GALAT: $*"; exit 1; }
usage() {
    cat <<USAGE
$SCRIPT_NAME v$VERSION - laporan kesehatan sistem
Penggunaan: $SCRIPT_NAME [OPSI]
    -d N   Ambang peringatan pemakaian disk dalam persen (default: 80)
    -j     Keluarkan hasil dalam format JSON
    -h     Tampilkan bantuan ini
Exit code: 0 = sehat, 2 = melewati ambang, 1 = galat penggunaan
USAGE
}
disk_usage_pct() { df -P / | awk 'NR==2 { gsub("%","",$5); print $5 }'; }
mem_used_pct() { free | awk '/^Mem:/ { printf "%.0f", $3/$2*100 }'; }
proc_count() { ps -e --no-headers | wc -l; }
main() {
    local disk mem procs status opt
    while getopts ":d:jh" opt; do
        case "$opt" in
            d) THRESHOLD_DISK="$OPTARG" ;;
            j) FORMAT="json" ;;
            h) usage; exit 0 ;;
            :) usage >&2; die "opsi -$OPTARG membutuhkan argumen" ;;
            \?) usage >&2; die "opsi tidak dikenal: -$OPTARG" ;;
        esac
    done
    [[ "$THRESHOLD_DISK" =~ ^[0-9]+$ ]] || die "ambang disk harus berupa angka"
    disk="$(disk_usage_pct)"; mem="$(mem_used_pct)"; procs="$(proc_count)"
    status="OK"; ((disk >= THRESHOLD_DISK)) && status="PERINGATAN"
    if [[ "$FORMAT" == "json" ]]; then
        printf '{"host":"%s","disk_pct":%s,"mem_pct":%s,"proc":%s,"status":"%s"}\n' \
            "$(hostname)" "$disk" "$mem" "$procs" "$status"
    else
        printf '%-20s : %s\n' 'Host' "$(hostname)"
        printf '%-20s : %s%\n' 'Pemakaian disk' "$disk"
        printf '%-20s : %s%\n' 'Pemakaian memori' "$mem"
        printf '%-20s : %s\n' 'Jumlah proses' "$procs"
        printf '%-20s : %s\n' 'Status' "$status"
    fi
    [[ "$status" == "OK" ]] || return 2
}
main "$@"
```

Lampiran B.3 — `setup.sh` (modular, Minggu 2)

```
#!/usr/bin/env bash
# setup.sh - versi MODULAR: fungsi bersama di lib/common.sh + logging terstruktur.
set -euo pipefail
APP_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
# shellcheck source=lib/common.sh disable=SC1091
source "$APP_DIR/lib/common.sh"
VENV_DIR="$APP_DIR/.venv"
PORT="${PORT:-5000}"
log_info "[1/5] Memeriksa prasyarat..."
require_cmd python3
require_cmd curl
python3 -c 'import sys; sys.exit(0 if sys.version_info >= (3, 10) else 1)' \
  || die "dibutuhkan Python 3.10 atau lebih baru"
port_is_free "$PORT" || die "port $PORT sudah dipakai proses lain"
log_info "[2/5] Menyiapkan virtual environment..."
[ -d "$VENV_DIR" ] || python3 -m venv "$VENV_DIR"
# shellcheck source=/dev/null
source "$VENV_DIR/bin/activate"
log_info "[3/5] Memasang dependensi terkunci..."
pip install --quiet --upgrade pip
pip install --quiet -r "$APP_DIR/requirements.txt"
log_info "[4/5] Menjalankan aplikasi pada port $PORT..."
PORT="$PORT" python3 "$APP_DIR/src/app.py" &
APP_PID=$!
trap 'kill "$APP_PID" 2>/dev/null || true' EXIT
log_info "[5/5] Melakukan smoke test..."
ok=0
for i in 1 2 3 4 5; do
  if curl -fsS "http://127.0.0.1:$PORT/health" >/dev/null 2>&1; then ok=1; break; fi
  log_warn " percobaan $i/5 belum berhasil, menunggu 2 detik..."
  sleep 2
done
[ "$ok" -eq 1 ] && log_info "SUKSES: lulus health check (PID $APP_PID, port $PORT)." || die "tidak merespons health check"
log_info "Aplikasi aktif di http://127.0.0.1:$PORT/ (Ctrl+C untuk berhenti)."
wait "$APP_PID"
```

Lampiran B.4 — lib/common.sh

```
#!/usr/bin/env bash
# lib/common.sh – Pustaka fungsi bersama (logging & validasi).
LOG_FILE="${LOG_FILE:-/tmp/sentra-deploy.log}"
_ts() { date '+%Y-%m-%dT%H:%M:%S%z'; }
log_info() { printf '%s [INFO ] %s\n' "$(_ts)" "$*" | tee -a "$LOG_FILE" >&2; }
log_warn() { printf '%s [WARN ] %s\n' "$(_ts)" "$*" | tee -a "$LOG_FILE" >&2; }
log_error() { printf '%s [ERROR] %s\n' "$(_ts)" "$*" | tee -a "$LOG_FILE" >&2; }
die() { log_error "$*"; exit 1; }
require_cmd() { command -v "$1" >/dev/null 2>&1 || die "perkakas wajib tidak ditemukan: $1"; }
port_is_free() { ! ss -ltn "sport = :$1" 2>/dev/null | grep -q LISTEN; }
```

Lampiran B.5 — healthwatch.sh (Challenge)

```
#!/usr/bin/env bash
# healthwatch.sh - CHALLENGE "Health Watcher": pantau /health, MTTR, ringkasan sesi.
set -euo pipefail
INTERVAL=5
URL="http://127.0.0.1:5000/health"
LOG_FILE="${LOG_FILE:-/tmp/healthwatch.log}"
CHECKS=0; INCIDENTS=0; TOTAL_DOWN=0; DOWN_SINCE=0; PREV_STATE="UP"
usage() {
    cat <<USAGE
healthwatch.sh - pemantau endpoint /health
--interval N    Jeda antar pemeriksaan (detik, default 5, > 0)
--url URL       URL dipantau (default $URL)
-h, --help      Bantuan
Exit: 0 tanpa insiden, 2 bila ada insiden
USAGE
}
die() { echo "GALAT: $*" >&2; exit 1; }
iso_now() { date '+%Y-%m-%dT%H:%M:%S%z'; }
while [[ $# -gt 0 ]]; do
    case "$1" in
        --interval) [[ $# -ge 2 ]] || die "--interval butuh nilai"; INTERVAL="$2"; shift 2 ;;
        --url)      [[ $# -ge 2 ]] || die "--url butuh nilai"; URL="$2"; shift 2 ;;
        -h|--help)  usage; exit 0 ;;
        *) usage >&2; die "argumen tidak dikenal: $1" ;;
    esac
done
[[ "$INTERVAL" =~ ^[0-9]+$ ]] || die "--interval harus bilangan bulat"
((INTERVAL > 0)) || die "--interval harus > 0"
[[ "$URL" =~ ^https?:// ]] || die "--url harus http(s)://"
print_summary() {
    echo; echo "==== RINGKASAN SESI HEALTH WATCHER ====="
    local avail="100.00"
    ((DOWN_SINCE > 0)) && TOTAL_DOWN=$((TOTAL_DOWN + ((date +%s) - DOWN_SINCE)))
    local total=$((CHECKS * INTERVAL))
    ((total > 0)) && avail=$(awk -v d="$TOTAL_DOWN" -v t="$total" 'BEGIN{printf "%.2f",(t-d)/t*100}')
    printf '%-24s : %s\n' "Jumlah pemeriksaan" "$CHECKS"
    printf '%-24s : %s\n' "Jumlah insiden" "$INCIDENTS"
    printf '%-24s : %s detik\n' "Total waktu padam" "$TOTAL_DOWN"
    printf '%-24s : %s%%\n' "Ketersediaan" "$avail"
    ((INCIDENTS > 0)) && exit 2 || exit 0
}
trap print_summary SIGINT
echo "Memantau $URL setiap ${INTERVAL}s - Ctrl+C untuk berhenti."; : > "$LOG_FILE"
while true; do
    s=$(date +%s)
    if curl -fsS --max-time "$INTERVAL" "$URL" >/dev/null 2>&1; then state="UP"; else state="DOWN"; fi
    e=$(date +%s); rt_ms=$((e-s)/1000000); CHECKS=$((CHECKS+1))
    printf '%s status=%s rt_ms=%s\n' "$(iso_now)" "$state" "$rt_ms" | tee -a "$LOG_FILE"
    if [[ "$state" == "DOWN" && "$PREV_STATE" == "UP" ]]; then
        DOWN_SINCE=$(date +%s); INCIDENTS=$((INCIDENTS+1))
        echo " !! INSIDEN #$INCIDENTS dimulai $(iso_now)" | tee -a "$LOG_FILE"
    elif [[ "$state" == "UP" && "$PREV_STATE" == "DOWN" ]]; then
        m=$(( (date +%s) - DOWN_SINCE )); TOTAL_DOWN=$((TOTAL_DOWN+m)); DOWN_SINCE=0
        echo " >> PULIH $(iso_now) - MTTR: ${m} detik" | tee -a "$LOG_FILE"
    fi
    PREV_STATE="$state"; sleep "$INTERVAL"
done
```

Lampiran E — Bukti & Tautan (diisi mahasiswa)

E.1 — ringkasan-log.txt (hasil nyata JOB 2)

```
=== Ringkasan Analisis Log 2026-10-04 07:22:40 ===
Total permintaan : 500
--- Sebaran kode status ---
    205 200 | 97 404 | 59 301 | 56 502 | 47 201 | 36 500
--- Endpoint paling sering gagal (≥500) ---
    30 /health | 23 /api/users | 21 /static/app.js | 18 /api/orders
Proksi Change Failure Rate: 18.40% (92 dari 500)
```

E.2 — Tautan & screenshot

Item	Isi
Tautan repository GitHub (private)	https://github.com/_____/praktikum-devops-m01
Collaborator dosen	Antoni Haikal (antoni@polibatam.ac.id) — ditambahkan: ☐

[Screenshot keluaran `shellcheck` bersih untuk ketiga skrip]

[Screenshot keluaran `sysreport.sh` + tabel exit code]

[Cuplikan `/tmp/sentra-deploy.log` & ringkasan Health Watcher]

— Selesai —